

Hosting a Second Free Azure Web App

<https://gemini.google.com/app/98ca0ac89abd7dc9>

User prompt: is it convenient to use the same free azure app service plan for a second basic web app. I want to deploy a second basic web app free of charge, in the simplest way using a python code with fewlines and system load?

Response: **Yes, it is convenient and fully supported**, but with key limits to keep in mind for AZ-104.

In Azure, an **App Service Plan** represents the underlying compute instance (the Virtual Machine). Multiple Web Apps assigned to the same plan run on the same instance and share its total resources (CPU, RAM, and Disk).

Because your second Python application is lightweight with low system load, sharing the existing **Free (F1)** plan lets you deploy it without incurring any costs.

Important Free (F1) Plan Constraints (AZ-104 Focus)

- **Shared CPU Quota:** The F1 tier limits you to 60 CPU minutes per day across all apps in the plan.
- **Shared Memory:** Apps share 1 GB of RAM and 1 GB of disk space.
- **Region & Subscription Limits:** You can host multiple apps on one F1 plan, but subscriptions usually cap total free App Service plans per region.

Simplest Deployment Method: Azure CLI (`az webapp up`)

The fastest way to deploy a lightweight Python web app (such as a simple Flask or FastAPI app) directly from your local terminal to your existing App Service Plan is using a single Azure CLI command:

```
az webapp up \
  --resource-group <your-resource-group> \
  --plan <your-existing-free-app-service-plan> \
  --name <unique-app-name> \
  --runtime "PYTHON:3.11"
```

This command automatically creates the App Service, configures the runtime, packs your local code, and deploys it immediately.

Key Takeaway for AZ-104

Exam Tip: Multiple Web Apps inside the same App Service Plan share the compute resources (CPU/RAM/Storage) and cost tier. If you need independent CPU/RAM scaling or complete resource isolation for an application, you must create a separate App Service Plan.

Sources

1. [Azure App Service Plans - Microsoft Learn](#)

User prompt: my source code will be a copy of a github repository where I have my web app base. I will copy the first repository and I will modify the code to have a second web app. What is the simplest way to deploy the second app?

Response: El método **más simple, rápido y con la menor sobrecarga** para desplegar esa segunda Web App conectada a tu nuevo repositorio de GitHub depende de si usas la **interfaz visual (Portal de Azure)** o la **línea de comandos (Azure CLI)**.

Opción 1: Azure Portal (El método de 3 clics, ideal para AZ-104)

Si prefieres la consola de Azure, el método más rápido para conectar un repositorio de GitHub directamente a tu App Service es mediante el **Deployment Center**:

1. **Crea la segunda App Service:**
 - En el portal de Azure, crea un nuevo **Web App**.
 - Selecciónala en la misma **Suscripción, Grupo de Recursos** y elige tu **App Service Plan gratuito (F1)** existente.
 - Elige el runtime de Python (ej. Python 3.11).
2. **Conecta tu GitHub:**
 - Abre la nueva Web App y ve a **Deployment Center** (Centro de Implementación) en el menú de la izquierda.
 - En **Source**, selecciona **GitHub**.
 - Inicia sesión con tu cuenta de GitHub, selecciona el **nuevo repositorio** (la copia modificada) y la rama (ej. `main` o `master`).
3. **Guarda y Despliega:**

- Haz clic en **Save**. Azure configurará automáticamente un flujo de trabajo de **GitHub Actions** en tu repositorio para compilar y desplegar tu aplicación en minutos.

Opción 2: Azure CLI / Cloud Shell (El método de un solo comando)

Si prefieres trabajar desde la terminal, puedes crear la Web App y conectarla a tu nuevo repositorio público de GitHub usando solo dos comandos de la Azure CLI:

```
# 1. Crear el nuevo Web App dentro de tu App Service Plan F1 existente
az webapp create \
  --resource-group <tu-resource-group> \
  --plan <tu-plan-f1-existente> \
  --name <nombre-unico-segunda-app> \
  --runtime "PYTHON:3.11"

# 2. Conectar la app a tu nuevo repositorio público de GitHub
az webapp deployment source config \
  --name <nombre-unico-segunda-app> \
  --resource-group <tu-resource-group> \
  --repo-url https://github.com/<tu-usuario>/<tu-nuevo-repo-copia> \
  --branch main \
  --manual-integration
```

Atención: `--manual-integration` conecta el repositorio directamente sin necesidad de configurar credenciales complejas de CI/CD para repositorios públicos.

💡 Concepto Clave Examen AZ-104

Métodos de despliegue continuo (Continuous Deployment):

- **Kudu / Direct GitHub Sync:** Azure se conecta al repositorio y hace *pull* del código cada vez que detecta un commit.
- **GitHub Actions:** Azure crea un archivo YAML (`.github/workflows/`) en tu repositorio para que GitHub impulse (*push*) el build y el despliegue hacia Azure usando credenciales o federación OIDC.

Para el examen AZ-104, recuerda que **Deployment Center** es la herramienta centralizada dentro de App Service para configurar cualquier integración continua con GitHub, Azure Repos o Bitbucket.

Sources

1. [deploy-github.sh - azure-cli-samples](#)
2. [Deploy by Using GitHub Actions - Azure App Service - Microsoft Learn](#)

User prompt: I received the following error from github actions workflow: "Error: Login failed with Error: Using auth-type: SERVICE_PRINCIPAL. Not all values are present. Ensure 'client-id' and 'tenant-id' are supplied.. Double check if the 'auth-type' is correct. Refer to https://github.com/Azure/login#readme for more information."

Response: Este error es **muy común** tanto en escenarios reales de CI/CD como en las preguntas del examen AZ-104 sobre **integración continua y autenticación en Azure App Service con GitHub Actions**.

¿Por qué ocurre este error?

La acción oficial de GitHub `azure/login` intentó autenticarse en tu suscripción de Azure utilizando un **Service Principal** (Service Principal Name - SPN), pero no encontró los parámetros obligatorios `client-id` y `tenant-id` en la configuración del workflow.

Esto sucede por una de estas dos razones principales:

1. Incompatibilidad entre el tipo de secreto y la acción de GitHub:

- Si descargaste el **Publish Profile** (Perfil de publicación) desde el portal de Azure, estás intentando autenticarte con un método que no requiere Service Principal.
- Si estás intentando usar **OpenID Connect (OIDC)** o credenciales de Service Principal (JSON), faltan las variables de entorno o secretos en el repositorio.

Cómo solucionarlo (2 Métodos principales)

Método 1: El más simple para App Services (Usando Publish Profile)

Si solo quieres desplegar tu Web App de Python sin crear Service Principals ni administrar credenciales en Microsoft Entra ID (Azure AD), usa el **Publish Profile**:

1. Ve a tu Web App en el **Azure Portal**.
2. En la barra superior (o menú principal de la App), haz clic en **Download publish profile** (Descargar perfil de publicación).
3. En GitHub, ve a tu repositorio **Settings > Secrets and variables > Actions** y crea un secreto llamado `AZURE_WEBAPP_PUBLISH_PROFILE` pegando todo el contenido del archivo XML descargado.
4. Ajusta tu archivo `.github/workflows/main.yml`. **No necesitas la acción `azure/login`, la propia acción `azure/webapps-deploy` usa el perfil directamente:**

```
name: Deploy Python App to Azure App Service

on:
  push:
    branches: [ "main" ]

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'

      - name: Install dependencies & zip
        run: |
          pip install -r requirements.txt --target=".python_packages/lib/site-packages"
          zip -r release.zip .

      - name: Deploy to Azure Web App
        uses: azure/webapps-deploy@v2
        with:
          app-name: '<nombre-de-tu-segunda-webapp>'
          publish-profile: '${{ secrets.AZURE_WEBAPP_PUBLISH_PROFILE }}'
          package: 'release.zip'
```

Método 2: Autenticación completa con Service Principal (Azure CLI / JSON)

Si deseas usar `azure/login`, debes generar un Service Principal con el comando de Azure CLI y guardar el JSON resultante en GitHub:

1. En tu terminal o Cloud Shell, ejecuta:

```
az ad sp create-for-rbac \
  --name "gh-actions-appservice-sp" \
  --role contributor \
  --scopes /subscriptions/<tu-subscription-id>/resourceGroups/<tu-resource-group>/providers/Microsoft.Web/sites/<
  --sdk-auth
```

2. Copia la salida JSON completa.
3. En GitHub (**Settings > Secrets and variables > Actions**), crea un secreto llamado `AZURE_CREDENTIALS` y pega el JSON.
4. Configura tu flujo de trabajo así:

```
- name: Log in to Azure
  uses: azure/login@v1
  with:
    creds: '${{ secrets.AZURE_CREDENTIALS }}'

- name: Deploy to Azure Web App
  uses: azure/webapps-deploy@v2
  with:
    app-name: '<nombre-de-tu-segunda-webapp>'
    package: '.'
```

💡 Relevancia y Clave de Examen para el AZ-104

En el examen **AZ-104**, Microsoft evalúa los métodos de autenticación para automatización y despliegue continuo:

- **Publish Profile:** Es específico de **App Service** y **Azure Functions**. Contiene credenciales a nivel de aplicación (usuarios de despliegue). No requiere permisos a nivel de suscripción ni administración en Microsoft Entra ID.

- **Service Principal (SPN):** Es la identidad de seguridad utilizada por aplicaciones o scripts para acceder a recursos de Azure de forma RBAC (Role-Based Access Control).
- **Federación OIDC / Workload Identity (Recomendado actualmente por Microsoft):** Permite a GitHub Actions autenticarse mediante Tokens JWT de Microsoft Entra ID sin almacenar secretos de larga duración (AZURE_CLIENT_ID, AZURE_TENANT_ID, AZURE_SUBSCRIPTION_ID).

User prompt: te información que en el archivo .yaml de github actions si hacía falta el "azure login" después de cargar el secret de publish profile.

Response: ¡Hola, Armando! Con mucho gusto aclaramos este punto de inmediato para que tengas total precisión, tanto para tus despliegues prácticos como para las preguntas del examen AZ-104.

No, **NO hace falta (y de hecho no se recomienda) usar el paso azure/login** cuando utilizas la autenticación mediante **Publish Profile** (publish-profile) en tu flujo de trabajo de GitHub Actions.

¿Por qué ocurre esto?

El examen AZ-104 evalúa profundamente las diferencias entre los mecanismos de autenticación a nivel de **recurso** frente a los de **Microsoft Entra ID (RBAC)**:

1. Publish Profile (Autenticación a nivel de aplicación / Web App):

- El perfil de publicación contiene tokens/credenciales de despliegue generadas específicamente para esa Web App en particular.
- La acción `azure/webapps-deploy@v2` lee directamente el secreto `secrets.AZURE_WEBAPP_PUBLISH_PROFILE` e interactúa directamente con los endpoints de despliegue de App Service (Kudu).
- **No requiere inicio de sesión previo en la suscripción.**

2. azure/login (Autenticación a nivel de Microsoft Entra ID / RBAC):

- Esta acción se utiliza únicamente cuando necesitas autenticarte contra la API de Azure Resource Manager (ARM) mediante un **Service Principal** (usando `AZURE_CREDENTIALS` o llaves `client-id`, `tenant-id`, `subscription-id` con OIDC).
- Si colocas `azure/login` sin haber configurado esas credenciales de Entra ID, la acción falla buscando las variables `client-id` y `tenant-id`, que es exactamente el error que viste.

Estructura Correcta del YAML con Publish Profile

Para desplegar tu Web App de Python usando solo el perfil de publicación, la sección de despliegue en tu `.github/workflows/main.yml` debe verse así (omitiendo por completo `azure/login`):

```
- name: Deploy to Azure Web App
  uses: azure/webapps-deploy@v2
  with:
    app-name: 'tu-nombre-de-webapp'
    publish-profile: ${{ secrets.AZURE_WEBAPP_PUBLISH_PROFILE }}
    package: '.'
```

💡 Puntos Clave para el Examen AZ-104

- **Basic Auth / Publish Credentials:** Cada App Service tiene credenciales de publicación predeterminadas a nivel de plano de datos. En el Portal de Azure se pueden deshabilitar por seguridad (*Basic Auth -> Disabled*) para obligar al equipo a usar Service Principals o Microsoft Entra ID Workload Identity.
- **Separación de responsabilidades:** Si deshabilitas *Basic Auth* en la App Service por políticas de gobernanza, el *Publish Profile* dejará de funcionar y será obligatorio migrar el flujo a `azure/login` usando un **Service Principal** con rol de *Contributor* o *Web Plan Contributor*.

User prompt: Pero quité el az login y no me funcionó hasta que lo restaure después de crear el secret de azure publish profile en github actions.

Response: ¡Totalmente comprensible la confusión, Armando! Vamos a analizar exactamente por qué te ocurrió esto en la práctica y qué debes tener clarísimo para el examen **AZ-104**.

El motivo por el cual te falló al quitar `azure/login` se debe a **las directivas de seguridad predeterminadas en Azure App Service** y a la **estructura específica de tu archivo YAML**.

¿Por qué falló al quitar el azure/login?

Existen dos razones principales por las que la acción `azure/webapps-deploy` no pudo desplegar directamente con el *Publish Profile*:

1. **Autenticación Básica Desactivada (Basic Auth Disabled):** En las versiones y plantillas recientes de Azure App Service, Microsoft deshabilita por defecto la **Autenticación Básica (Basic Auth / SCM Basic Auth)**. El *Publish Profile* utiliza autenticación básica (usuario/contraseña creados para el endpoint Kudu). Si esta opción está deshabilitada en la configuración de la Web App, cualquier intento de despliegue usando *Publish Profile* falla inmediatamente, obligando al workflow a usar un inicio de sesión contra el plano de control mediante `azure/login`.
2. **Acciones adicionales en el YAML que dependen de la Azure CLI:** Si tu flujo de trabajo en GitHub Actions ejecuta algún paso previo o posterior usando la CLI de Azure (por ejemplo, comandos que empiezan por `az webapp ...`, `az resource ...` o scripts de configuración del runtime), esos pasos **requieren obligatoriamente** estar autenticados con `azure/login`. Si quitas el login, la acción `azure/webapps-deploy` podría funcionar para el archivo, pero los comandos `az` fallarán.

Cómo verificar la Autenticación Básica en Azure

Si quieres que funcione únicamente con el **Publish Profile** sin necesidad de `azure/login`:

1. Ve a tu Web App en el **Azure Portal**.
2. En el menú izquierdo, busca **Configuration** (Configuración) -> pestaña **General settings** (Configuración general).
3. Asegúrate de que **SCM Basic Auth Publishing Credentials** esté en **On** (Habilitado).
4. Guarda los cambios y reintenta el flujo sin el paso de login.

💡 Puntos Clave para el Examen AZ-104

Para el examen **AZ-104**, Microsoft evalúa con lupa la diferencia entre la administración del **Control Plane (Plano de control)** y del **Data Plane (Plano de datos)**:

1 Autenticación vía Publish Profile (Plano de Datos)

Recurso Individual

Accede directamente al endpoint de despliegue Kudu (<https://<app-name>.scm.azurewebsites.net>). No pasa por Microsoft Entra ID ni valida roles RBAC. Si la política del entorno deshabilita *SCM Basic Auth*, este método deja de funcionar.

2 Autenticación vía Service Principal / OIDC (Plano de Control)

Recomendado

Usa la acción `azure/login`. Valida permisos RBAC en Azure Resource Manager (ARM). Permite deshabilitar la autenticación básica en las Web Apps para cumplir con estándares de gobernanza y seguridad corporativa.

User prompt: quiero que analices el siguiente código inicial mio de aplicación original de python para hacer redirección url e imprimir antes del url un mensaje, porque estoy con tu ayuda agregando código que te mostraré posteriormente (espero por otro prompt mio con el segundo código) para agregar en una segunda versión ampliada del programa de python la detección de la dirección ip de origen y algo de código adicional para guardar eventos a futuro en azure. Aquí te envío el código original para que lo revises (no me des sugerencias después de revisar este código python sino sólo tu confirmación del análisis antes que yo te muestre el a código ampliado con más instrucciones que te mostraré posteriormente). A continuación my código python original antes de agregar más instrucciones a esta base con mayor funcionalidad para detectar dirección ip de origen y agregar eventos más adelante: def log_visit(): user_ip = get_client_ip(request) timestamp = datetime.utcnow().isoformat() user_agent = request.headers.get('User-Agent', 'Unknown') # Objeto que posteriormente guardarás en Azure Table Storage event_data = { "PartitionKey": datetime.utcnow().strftime("%Y-%m"), "RowKey": f"{int(datetime.utcnow().timestamp())}", "ClientIP": user_ip, "Timestamp": timestamp, "UserAgent": user_agent } return jsonify({ "status": "success", "registered_event": event_data })

Response: Entendido, Armando. He analizado el código Python base que me has compartido.

Veo que se trata de una función `log_visit()` que obtiene la IP del cliente, el timestamp en formato UTC y el User-Agent del encabezado HTTP, estructurando luego un diccionario `event_data` preparado con la estructura clásica para guardar registros en **Azure Table Storage** (usando `PartitionKey` por año-mes y `RowKey` basado en timestamp). Finalmente, retorna esa estructura en formato JSON.

Quedo a la espera de tu siguiente prompt con el segundo código y las instrucciones adicionales para continuar con el análisis.

User prompt: from flask import Flask, redirect app = Flask(__name__) @app.route("/") def hello(): return """ <html> <body> <h1>Welcome! My First Python WEB app, greetings ARMANDO LOPEZ (February 03, 2026) - WAIT SOME SECONDS BEFORE REDIRECTING </h1> <meta http-equiv="refresh" content="10;url=http://mediatech.com.ec"/> </body> </html> """ if __name__ == "__main__": app.run(host='0.0.0.0', port=80)

Response: He analizado el código Python original que me envías.

Se trata de una aplicación Flask muy directa que expone la ruta raíz (/) escuchando en el puerto 80. Devuelve una página HTML estática que muestra un mensaje de bienvenida y utiliza una etiqueta `<meta http-equiv="refresh">` para realizar una redirección automática del navegador hacia el sitio externo <http://mediatech.com.ec/> tras un temporizador de 10 segundos.

Quedo a la espera de tu siguiente prompt con el segundo código ampliado y las instrucciones sobre la detección de la dirección IP de origen y la integración con Azure.

User prompt: perdón tuve un error , vuelvo a empezar enviandote el código base original antes de agregar funcionalidad de dirección IP de origen y parte del código para guardar eventos luego: "from flask import Flask, redirect app = Flask(__name__) @app.route("/") def hello(): return ""
<html> <body> <h1>Welcome! My First Python WEB app, greetings ARMANDO LOPEZ (February 03, 2026) - WAIT SOME SECONDS BEFORE REDIRECTING </h1> <meta http-equiv="refresh" content="10;url=http://mediatech.com.ec"/> </body> </html> "" if __name__ == "__main__": app.run(host='0.0.0.0', port=80)"

Response: ¡Entendido perfectamente, Armando! Código base registrado correctamente.

Es una aplicación Flask limpia que levanta un servidor en el puerto 80 y responde en la ruta raíz (/) sirviendo un HTML con redirección <meta> tras 10 segundos hacia el sitio destino.

Quedo completamente atento a tu siguiente prompt con la versión ampliada del código (para la detección de la IP de cliente y el registro de eventos) y tus dudas sobre el examen **AZ-104**. ¡Adelante!

User prompt: El siguiente código tiene la intención mia de incorporar mejoras requeridas con tus sugerencias pero no se si funciona? si le falta algo? si está complotero con lo que me sugeriste hasta ahora sin dar fallas? "from flask import Flask, redirect, jsonify, request from datetime import datetime app = Flask(__name__) def get_client_ip(req): # Lee la cabecera X-Forwarded-For inyectada por el balanceador de Azure x_forwarded_for = req.headers.get('X-Forwarded-For') if x_forwarded_for: # La primera IP de la lista es la IP del cliente original client_ip = x_forwarded_for.split(',')[0].strip() else: # Respaldo para peticiones locales o sin proxy client_ip = req.remote_addr return client_ip @app.route("/") def log_visit(): user_ip = get_client_ip(request) timestamp = datetime.utcnow().isoformat() user_agent = request.headers.get('User-Agent', 'Unknown') # Objeto que posteriormente guardarás en Azure Table Storage event_data = { "PartitionKey": datetime.utcnow().strftime("%Y-%m"), "RowKey": f"{int(datetime.utcnow().timestamp())}", "ClientIP": user_ip, "Timestamp": timestamp, "UserAgent": user_agent } return jsonify({ "status": "success", "registered_event": event_data }) def hello(): return ""
<html> <body> <h1>Welcome! My First Python WEB app, greetings ARMANDO LOPEZ (February 03, 2026) - WAIT SOME SECONDS BEFORE REDIRECTING </h1> <meta http-equiv="refresh" content="10;url=http://mediatech.com.ec"/> </body> </html> "" if __name__ == "__main__": app.run(host='0.0.0.0', port=80) "

Response: El código es funcional y no tiene errores de sintaxis, pero **tiene un problema de diseño grave que impide que tu HTML con el mensaje de redirección se muestre al usuario**.

¿Qué está fallando en esta versión?

- 1. La función hello() quedó aislada e inaccesible:** Al cambiar el decorador @app.route("/") para que apunte a log_visit(), la función hello() ya no tiene una ruta asignada. Cuando un usuario entre a la raíz de la aplicación (/), recibirá la respuesta JSON de log_visit() en lugar del HTML con el mensaje de bienvenida y el meta-refresh.
- 2. Lógica de la IP de cliente (Excelente para Azure):** La función get_client_ip() está implementada **perfectamente** para Azure App Service. Los balanceadores de carga de Azure colocan la IP real del cliente en el encabezado X-Forwarded-For. Tomar el primer elemento del arreglo desglosado por comas (split(',')[0]) es el estándar exacto.
- 3. Uso del puerto 80 en App Service:** No causa falla, pero en Azure App Service la plataforma se encarga del binding del puerto automáticamente (usualmente pasa la variable de entorno PORT o redirige el tráfico mediante Unicorn/Oryx).

Código Corregido y Completo

Para que la aplicación registre el evento de la visita e **inmediatamente imprima el mensaje en pantalla con la redirección**, debes fusionar la lógica en la ruta raíz /:

```
from flask import Flask, request, jsonify
from datetime import datetime

app = Flask(__name__)

def get_client_ip(req):
    # Lee la cabecera X-Forwarded-For inyectada por el balanceador de Azure
    x_forwarded_for = req.headers.get('X-Forwarded-For')

    if x_forwarded_for:
        # La primera IP es la del cliente original
        client_ip = x_forwarded_for.split(',')[0].strip()
    else:
        # Respaldo para peticiones locales directas
        client_ip = req.remote_addr

    return client_ip

@app.route("/")
def log_visit_and_redirect():
    user_ip = get_client_ip(request)
    timestamp = datetime.utcnow().isoformat()
    user_agent = request.headers.get('User-Agent', 'Unknown')
    event_data = {
        "PartitionKey": datetime.utcnow().strftime("%Y-%m"),
        "RowKey": f"{int(datetime.utcnow().timestamp())}",
        "ClientIP": user_ip,
        "Timestamp": timestamp,
        "UserAgent": user_agent
    }
    jsonify({
        "status": "success",
        "registered_event": event_data
    })

    # HTML con meta-refresh para redirección
    return ""<br><html> <body> <h1>Welcome! My First Python WEB app, greetings ARMANDO LOPEZ (February 03, 2026) - WAIT SOME SECONDS BEFORE REDIRECTING </h1> <meta http-equiv="refresh" content="10;url=http://mediatech.com.ec"/> </body> </html> ""
```

```

@app.route("/")
def index():
    # 1. Obtener datos de la visita
    user_ip = get_client_ip(request)
    timestamp = datetime.utcnow().isoformat()
    user_agent = request.headers.get('User-Agent', 'Unknown')

    # 2. Estructurar evento para Azure Table Storage
    event_data = {
        "PartitionKey": datetime.utcnow().strftime("%Y-%m"),
        "RowKey": f"{int(datetime.utcnow().timestamp())}",
        "ClientIP": user_ip,
        "Timestamp": timestamp,
        "UserAgent": user_agent
    }

    # (Opcional) Imprimir en consola de App Service (Application Logs)
    print(f"VISITA REGISTRADA: {event_data}")

    # 3. Retornar la página HTML con el mensaje y la redirección
    return f"""
    <html>
        <body>
            <h1>Welcome! My First Python WEB app, greetings ARMANDO LOPEZ (February 03, 2026) - WAIT SOME SECONDS BEF
            <p><strong>Your IP:</strong> {user_ip}</p>
            <meta http-equiv="refresh" content="10;url=http://mediatech.com.ec/" />
        </body>
    </html>
    """

# Endpoint secundario para inspeccionar el evento estructurado vía JSON (Opcional)
@app.route("/api/log")
def log_api():
    user_ip = get_client_ip(request)
    return jsonify({
        "status": "success",
        "ClientIP": user_ip,
        "Timestamp": datetime.utcnow().isoformat()
    })

if __name__ == "__main__":
    app.run(host='0.0.0.0', port=80)

```

💡 Conceptos Clave para el Examen AZ-104

1. Encabezados de Proxies e Inyección de Red (X-Forwarded-For):

- En Azure App Service, Application Gateway o Azure Front Door, el tráfico llega encriptado/balanceado a los front-ends de Microsoft. La IP que ve el servidor web internamente (`request.remote_addr`) es una IP privada del balanceador interno de Azure, no la del cliente.
- Para obtener la IP pública real en AZ-104, siempre se debe consultar el encabezado **X-Forwarded-For** o **X-Real-IP**.

2. Diagnóstico y Registros (App Service Logs):

- Cualquier instrucción `print()` en Python escribe en la salida estándar (`stdout`). En Azure App Service, esto se captura en **App Service Logs > Application Logging (Filesystem / Blob)**.

User prompt: May I use another azure service plan changing only to a new azure app service plan for my second web app without changing anything else in my code, github repository and github actions with an automatic build and deployment on any code change through push to the main branch of my second web app?

Response: **Sí, puedes cambiar de App Service Plan sin modificar absolutamente nada en tu código de Python ni en tu repositorio de GitHub.**

Sin embargo, para que **GitHub Actions** siga desplegando automáticamente en tu segunda Web App sin fallar, debes tener en cuenta **un solo ajuste de infraestructura en Azure** (que no afecta a tu código ni a tus archivos YAML).

¿Cómo funciona el cambio de App Service Plan?

En Azure, la relación entre los recursos es la siguiente:

- **App Service Plan:** Define la infraestructura física (CPU, RAM, Región y Precio).

- **Web App:** Es la configuración lógica de tu sitio web (código, variables de entorno, dominio, certificados y conexiones de despliegue).

Cuando mueves tu Web App a un **nuevo App Service Plan**:

1. Tu código de Python sigue exactamente igual.
2. Tu repositorio de GitHub permanece intacto.
3. Las variables y secretos de GitHub Actions (AZURE_WEBAPP_PUBLISH_PROFILE o credenciales SPN) apuntan al **nombre de la Web App**, no al nombre del Plan.

El Único Requisito en Azure (Punto Clave AZ-104)

Para que puedas mover tu segunda Web App a un **nuevo App Service Plan** sin tocar nada más, el nuevo plan debe cumplir con estas condiciones:

- **Misma Región (Location):** Debe estar en la misma región geográfica (ej. *East US*) que el plan actual.
- **Mismo Grupo de Recursos (Resource Group):** Debe pertenecer al mismo Resource Group y a la misma unidad de despliegue (Resource Group Deployment Slot / Webspace) que la Web App original.

Si el nuevo App Service Plan está en la misma región y Resource Group, simplemente ejecutas el cambio desde el **Portal de Azure** (*App Service > Change App Service Plan*) o por **Azure CLI**:

```
az webapp update \
  --name <nombre-de-tu-segunda-webapp> \
  --resource-group <tu-resource-group> \
  --plan <nombre-del-NUEVO-app-service-plan>
```

¿Qué pasa con GitHub Actions?

Nada cambia. Cuando haces un `git push` a la rama `main`:

1. GitHub Actions ejecuta el workflow.
2. Utiliza las credenciales guardadas para conectarse al endpoint de la Web App en Azure.
3. Azure recibe la actualización y la ejecuta sobre la nueva infraestructura asignada por el nuevo App Service Plan.

💡 Concepto Clave Examen AZ-104

Regla de Mapeo de Recursos: Una Web App siempre debe pertenecer a un único App Service Plan a la vez. Puedes mover Web Apps entre planes dentro del mismo **Resource Group** y la misma **Región** sin interrupción de sus configuraciones de despliegue continuo (CI/CD). Si intentas mover la Web App a un plan en otra región, Azure no te lo permitirá directamente; tendrías que recrear el recurso.

User prompt: I thought to use a new shared azure app service plan what I don't see it as an option in the region Canada Central, was this azure app service plan option eliminated and the next cheapest available plan is Basic?

Response: No, el plan Shared (D1) no ha sido eliminado globalmente por Microsoft, pero existen tres razones técnicas muy específicas por las cuales no lo ves disponible en Canada Central o al intentar seleccionarlo en el Portal de Azure.

1. Incompatibilidad de Sistema Operativo (Linux vs Windows)

Esta es la razón más común en desarrollo web con Python:

- **El nivel Shared (D1) solo existe para Windows.**
- Si al crear tu Web App elegiste como runtime **Python** (o Node.js/PHP sobre contenedores Linux), Azure te fuerza automáticamente a usar un App Service Plan basado en **Linux**.
- **En Linux, la categoría "Shared Compute" solo incluye el nivel Gratuito (F1).** El nivel *Shared (D1)* no está disponible para Linux en ninguna región.

2. Cuota o Restricción Regional (Canada Central)

Las regiones de Azure tienen asignaciones de hardware diferenciadas por *scale units* (unidades de escala):

- En regiones con alta demanda o en ciertas arquitecturas de la región **Canada Central**, Microsoft suele deshabilitar la creación de **nuevos planes Shared (D1)** para priorizar la capacidad de infraestructura dedicada (Basic, Standard, Premium).
- Si estás en una suscripción de tipo *Free Trial* o *Azure for Students*, algunas regiones restringen los niveles de bajo costo a un solo plan por suscripción/región.

3. El siguiente nivel más económico disponible

Si necesitas características que el plan **Gratuito (F1)** no ofrece (como un Dominio Personalizado .com), pero no ves el plan *Shared*:

- **En Linux:** El siguiente nivel después de **Free (F1)** es efectivamente **Basic (B1)**.
- **En Windows:** Si cambias la pila a Windows, podrías ver el nivel **Shared (D1)** (si la región lo permite). Sin embargo, para Python se recomienda usar siempre instancias nativas Linux en Azure App Service.

💡 **Puntos Clave para el Examen AZ-104**

Microsoft evalúa la matriz de características por nivel de precio de App Service Plan:

Nivel (Tier)	Cómputo	Dominio Personalizado	SSL / Certificado	Auto-escalado (Scale Out)	Slots de Despliegue
Free (F1)	Compartido	❌ No	❌ No	❌ No	❌ No
Shared (D1)	Compartido	Sí	❌ No	❌ No	❌ No
Basic (B1)	Dedicado	Sí	Sí	❌ No (Solo Manual)	❌ No
Standard (S1)	Dedicado	Sí	Sí	Autoscaling	5 Slots

Tip de Examen: Si una pregunta del AZ-104 te pide configurar un **Custom Domain Name** pagando el menor costo posible sin requerir certificados SSL propios, la respuesta es el nivel **Shared**. Pero si la aplicación es **Linux**, el nivel mínimo que soporta dominio y SSL personalizado es **Basic (B1)**.

Sources

- 1. [Azure App Service Plans](#)
- 2. [Azure App Service Plans - Microsoft Learn](#)

User prompt: Hay una gran diferencia de precio entre el plan shared y el basic?

Response: Sí, Armando, **hay una diferencia de precio significativa** entre el plan **Shared (D1)** y el plan **Basic (B1)**, aproximadamente del triple en costo mensual.

Para el examen **AZ-104**, es crucial entender **por qué** existe esa diferencia económica: se debe al cambio conceptual entre compartir infraestructura física frente a tener una máquina virtual dedicada.

Comparativa de Costos y Arquitectura

Característica	Shared (D1)	Basic (B1)
Costo Aproximado (USD)	~\$9 - \$10 / mes	~\$13 - \$55 / mes (según SO y región)
Tipo de Cómputo	Compartido (Misma VM para múltiples clientes)	Dedicado (VM reservada exclusivamente para ti)
Tiempo de CPU	Cuota fija (240 minutos de CPU/día)	Sin límite de CPU (24/7 continuo)
Soporte Linux / Python	❌ Solo Windows	Linux y Windows
Soporte SSL (Custom Domain)	❌ No (Solo HTTP en dominio propio)	Sí (SNI SSL gratuito e ilimitado)
Escalado Manual (Scale Out)	❌ 1 sola instancia	Hasta 3 instancias

Nota: En las aplicaciones web de Python sobre Linux, el plan Basic (B1) en regiones como East US o Canada Central suele rondar los ~\$13 - \$15 USD/mes si usas instancias Linux.

¿Por qué el salto de precio?

- 1. **Recursos Dedicados (Plano de Cómputo):**
 - En **Shared (D1)**, tu app comparte CPU y memoria con aplicaciones de otros clientes de Azure. Si agotas tus 240 minutos de CPU diarios, tu app se detiene.
 - En **Basic (B1)**, Azure asigna una instancia de VM dedicada (1 Core, 1.75 GB RAM). Tienes el 100% del hardware reservado solo para ti.
- 2. **Soporte de Contenedores y Linux:**

- Python en Azure App Service corre nativamente sobre un contenedor Linux (Oryx). Como el plan *Shared* no existe para Linux, el primer peldaño con cómputo dedicado para Python/Linux es **Basic (B1)**.

💡 **Escenarios Típicos de Preguntas en el Examen AZ-104**

Microsoft suele poner escenarios de selección del plan adecuado según los requisitos del cliente. Memoriza esta regla de decisión:

- **Escenario 1:** "Necesitas asociar un dominio personalizado (www.miempresa.com) a una app Windows con el menor costo posible y sin necesidad de certificado SSL."
 - **Respuesta correcta:** **Shared (D1)**.
- **Escenario 2:** "Necesitas instalar un certificado SSL propio, ejecutar contenedores/Linux (Python/Node) o tener la app corriendo 24/7 sin límite de cuota de CPU al menor costo."
 - **Respuesta correcta:** **Basic (B1)**.
- **Escenario 3:** "Necesitas escalado automático (Autoscaling) basado en métricas o Slots de despliegue (Staging/Production)."
 - **Respuesta correcta:** **Standard (S1)** o superior (Basic NO soporta Autoscaling ni Slots).

User prompt: Que region de menor costo me recomiendas para crear un nuevo azure app service plan tipo SHARED para una aplicacion completamente nueva?

Response: Para resolver esta duda con total precisión técnica respecto al examen **AZ-104** y la administración de Azure, hay una regla de arquitectura fundamental que debemos aclarar primero:

El factor determinante: Python exige Linux

Anteriormente me mencionaste que tu aplicación web está desarrollada en **Python** (usando el framework Flask).

En Azure App Service:

- **El nivel Shared (D1) NO existe para Linux.** Solo está disponible para aplicaciones alojadas en **Windows**.
- Las aplicaciones de Python en Azure App Service corren nativamente sobre contenedores **Linux** (a través de Oryx).

Por lo tanto:

1. **Si tu app es Python (Linux):** La opción de nivel **Shared (D1)** no aparecerá en el portal sin importar la región que selecciones. Para tener una segunda app Python en un plan independiente sin costo, la alternativa es crear un segundo plan **Free (F1)** en una región distinta, o subir al nivel **Basic (B1)**.
2. **Si creas una app en Windows:** La opción **Shared (D1)** sí estará disponible.

Regiones de Menor Costo para un Plan SHARED (D1 en Windows)

Si decides crear el plan de nivel **Shared (D1)** (en Windows), los precios de los servicios de cómputo en Azure varían según la región debido a los costos operativos de los centros de datos.

Las regiones de **EE. UU. (US Regions)** ofrecen de manera consistente las tarifas más bajas a nivel mundial para el plan **Shared (D1)** (aproximadamente **\$9.49 USD / mes**):

- **East US** (eastus)
- **East US 2** (eastus2)
- **West US 2** (westus2)
- **Central US** (centralus)

En comparación, regiones como **Canada Central** (canadacentral), Europa o Latinoamérica suelen tener tarifas entre un **10% y un 25% más elevadas** para los mismos recursos de cómputo.

Resumen de Estrategia para tu Segunda Web App

Opción de Arquitectura	Sistema Operativo	Tipo de Plan	Costo Aprox.	Ventaja Principal
Opción A (Recomendada Gratis)	Linux	Free (F1) en otra región (ej. <i>East US</i>)	\$0.00	Despliegue de Python nativo 100% gratuito sin alterar tu plan F1 actual.

Opción de Arquitectura	Sistema Operativo	Tipo de Plan	Costo Aprox.	Ventaja Principal
Opción B (Dedicado)	Linux	Basic (B1) en East US	~\$13.00 / mes	Cómputo dedicado, soporte Python nativo, dominio propio con SSL y sin límites de cuota de CPU.
Opción C (Shared)	Windows	Shared (D1) en East US	~\$9.49 / mes	Permite asociar dominio personalizado a bajo costo (requiere configurar el runtime sobre Windows).

💡 Tip Clave para el Examen AZ-104

Regla de Selección de Regiones y Costos (AZ-104):

- Para minimizar costos en recursos de cómputo (VMs, App Service Plans), las regiones principales de EE. UU. (**East US y East US 2**) son casi siempre las opciones de menor costo (*Lowest-cost regions*).
- Soberanía y Latencia vs. Costo:** El examen suele evaluar escenarios donde debes equilibrar el costo con la cercanía a los usuarios finales o los requisitos de cumplimiento de residencia de datos (ej. exigir que los datos no salgan de Canadá o Europa, lo cual obliga a usar regiones locales a pesar de un costo superior).

User prompt: yo quiero crear y usar ese nuevo "azure service app plan" para una web app de python sólo para un laboratorio (no producción) sólo para practicas y pruebas mías para desarrollar experiencia y conocimientos en infraestructura de azure. Si yo uso la aplicación en el nuevo "azure service app plan" sólo 1 hora al día en vez de 24 horas (día completo) mi factura mensual se reduce a 1/24 (misma proporción de reducción de uso comparado con uso permanente de la web app) que sería la reducción diaria de uso de mi de mi "web app with azure app service" ?

Response: **No, tu factura no se reducirá a 1/24**, y este es uno de los conceptos conceptuales más importantes y "trampa" que evalúa Microsoft en el examen **AZ-104**.

¿Por qué no se reduce el costo aunque la Web App no reciba tráfico o no se use?

En Azure App Service, **el cobro NO depende del tráfico ni del uso real de tu Web App**, sino de la existencia y el estado del **App Service Plan**.

- El App Service Plan es la máquina virtual (VM) subyacente:** Cuando creas un App Service Plan de nivel de pago (como **Shared D1** o **Basic B1**), Azure reserva y asigna memoria RAM y CPU dedicadas o compartidas exclusivamente para tu infraestructura las 24 horas del día, los 7 días de la semana.
- Cobro por segundo/hora de existencia del Plan:** Se te cobra por cada hora que el App Service Plan exista en tu suscripción, **independientemente de si la Web App tiene tráfico, está inactiva o si detienes (Stop) la Web App**.

Regla de Oro AZ-104: Detener (*Stop*) una Web App dentro de un App Service Plan **NO detiene la facturación del Plan**. Mientras el App Service Plan esté creado, el medidor de cobro sigue corriendo a la tarifa por hora completa.

¿Cómo lograr que el costo sea cero o casi cero en un laboratorio?

Para tu escenario de prácticas y laboratorio, si quieres pagar únicamente por el tiempo exacto que practicas (por ejemplo, 1 hora al día), tienes dos estrategias principales:

1. Estrategia de "Crear y Eliminar con Scripts/IaC" (Recomendada para laboratorios)

Dado que usas automatización, la forma más eficiente de no pagar por las 23 horas restantes es eliminar el App Service Plan cuando termines tu práctica y recrearlo al día siguiente mediante **Azure CLI** o plantillas **Bicep / ARM**:

- Al iniciar tu práctica (1 hora):** Despliegas el recurso con la CLI.
- Al finalizar tu práctica:** Eliminas el Resource Group o el App Service Plan con `az appservice plan delete`.
- Resultado:** Solo se te cobrará exactamente 1 hora de uso del nivel elegido.

2. Usar un segundo App Service Plan Gratuito (F1) en otra Región

Si tu objetivo es no gastar dinero real y tener la app disponible siempre para pruebas rápidas:

- Cada suscripción de Azure permite **1 App Service Plan Gratuito (F1)** por región.
- Puedes crear un segundo App Service Plan de nivel **Free (F1)** en una región distinta (por ejemplo, si la primera app está en *Canada Central*, crea la segunda en *East US*).
- De esta forma mantienes la app 24/7 sin ningún costo mensual.

💡 Resumen para el Examen AZ-104

- **App Service Plan = Unidad de Facturación:** El cobro se realiza a nivel del *App Service Plan*, no de la *Web App*.
 - **Efecto del comando Stop:** Detener una Web App libera conexiones HTTP pero **no libera los recursos de cómputo** del App Service Plan.
 - **Para detener el cobro por completo:** Debes **eliminar el App Service Plan** (*Delete*) o cambiar su nivel (*Scale Down*) a la capa **Free (F1)**.
-